초급반 5주차

이O재 @ AlKor, CYDF

목차

- 복습
- 함수
- 구조체
- 공용체
- 열거형
- 연습문제

복습 - 2566

```
#include <bits/stdc++.h>

int main() {
    int a[9][9], maxi, maxj, maxval = -1;
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            scanf("%d", &a[i][j]);
            if (a[i][j] > maxval) {
                maxval = a[i][j];
                maxi = i;
                maxj = j;
            }
        }
    }

    printf("%d\n%d %d", maxval, maxi+1, maxj+1);
}
```

복습 - 2563

```c
#include <bits/stdc++.h>

int main() {
    int a[101][101] = {0, }, n, ans = 0;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        int x, y;
        scanf("%d %d", &x, &y);

        for (int j = x; j < x + 10; j++) {
            for (int k = y; k < y + 10; k++) {
                a[j][k] = 1;
            }
        }
    }
```

```c
    for (int i = 1; i <= 100; i++) {
        for (int j = 1; j <= 100; j++) {
            ans += a[i][j];
        }
    }
    printf("%d", ans);
}
```

복습 - 2446

```c
#include <bits/stdc++.h>

int main() {
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            printf(" ");
        }
        for (int j = 0; j < 2*(n-i)-1; j++) {
            printf("*");
        }
        printf("\n");
    }
```

```c
    for (int i = n-2; i >= 0; i--) {
        for (int j = 0; j < i; j++) {
            printf(" ");
        }
        for (int j = 0; j < 2*(n-i)-1; j++) {
            printf("*");
        }
        printf("\n");
    }
}
```

함수(복습)

- function: 기능, 기능을 담당하는 부분을 따로 떼어내서 구현할 때 사용

```
for (int i = m; i <= n; i++) {
    if (isPrime(i)) {
        if (prime == 0) prime = i;
        ans += i;
    }
}
```

- 함수의 정의

```
<type> <name>(<parameter(s)>) {  }

int isPrime(int number) {
    // 코드 내용
    return 0;
}
```

함수(복습)

- 함수의 반환 값
  - 함수는 자신의 type에 맞는 값을 반환해야 함

```
#include <bits/stdc++.h>

int a() {
    return 1.3;
}

int main() {
    printf("%f\n", (float) a()); // 1.000000
}
```

  - 만약 아무런 값도 반환하고 싶지 않으면 void로 선언함

```
void a() {
    // code
    return; // 있어도 되고 없어도 됨
}
```

함수(복습)

- 함수는 값을 반환하는 즉시 종료됨

```
int isPrime(int number) {
    if (number <= 1) return 0;

    for (int i = 2; i * i <= number; i++) {
        if (number % i == 0) return 0;
    }

    return 1;
}
```

함수(복습)

- 함수의 종류

- 사용자 정의 함수
  - 일반적으로 의미하는 함수

- 라이브러리 함수
  - `printf`, `scanf` 등 사전에 정의된 함수

함수(복습)

- 함수의 선언
  - 함수는 항상 main 함수 앞에 선언하며, 선언과 동시에 구현해도 됨

```
#include <bits/stdc++.h>

int isPrime(int number);

int main() {
    printf("%d", isPrime(13));
}

int isPrime(int number) {
    // 구현 시작
}
```

```
#include <bits/stdc++.h>

int isPrime(int number) {
    // 구현 시작
}

int main() {
    printf("%d", isPrime(13));
}
```

함수(복습)

- 함수의 선언에서 헷갈릴 만한 점 ➔ 출력 결과 예측해보기

```
#include <bits/stdc++.h>

void f(int a, int b);

int main() {
    f(10, 20);
}

void f(int b, int a) {
    printf("%d\n", a);
    printf("%d\n", b);
}
```

함수(복습)

- 함수의 선언에서 헷갈릴 만한 점 ➔ 출력 결과 예측해보기

```
#include <bits/stdc++.h>

void f(int a, int b);

int main() {
    f(10, 20);
}

void f(int b, int a) {
    printf("%d\n", a);
    printf("%d\n", b);
}
```

```
20
10
```

함수(복습)

- 전역변수와 지역변수
  - 전역변수: 블록의 외부공간에 선언되는 변수
  - 지역변수: 블록의 내부공간에 선언되는 변수 ➔ 블록의 내부에서만 사용 가능

```
#include <bits/stdc++.h>

int x = 10;

int a() {
    int y = 20;
}

int main() {
    printf("%d\n", x); // 10
    printf("%d\n", y); // error
}
```

함수(복습)

- 전역변수와 지역변수의 우선순위
  - 항상 지역변수가 우선적으로 고려됨

```
#include <bits/stdc++.h>

int x = 10;

void f(int x, int y) {
    x += 10;
    y += 30;
    printf("%d %d\n", x, y);
}

int main() {
    f(20, 30);          // 30 60
    printf("%d\n", x); // 10
}
```

함수(복습)

- 더 나아가, for문에서도 이와 유사하게 작동함

```
#include <bits/stdc++.h>

int x = 100;

int main() {
    int x = 10;

    for (int i = 0; i < 1; i++) {
        int x = 1;
        printf("%d\n", x);
    }

    printf("%d\n", x);
}
```

함수(복습)

- 더 나아가, for문에서도 이와 유사하게 작동함

```
#include <bits/stdc++.h>

int x = 100;

int main() {
    int x = 10;

    for (int i = 0; i < 1; i++) {
        int x = 1;
        printf("%d\n", x);
    }

    printf("%d\n", x);
}
```

```
1
10
```

함수

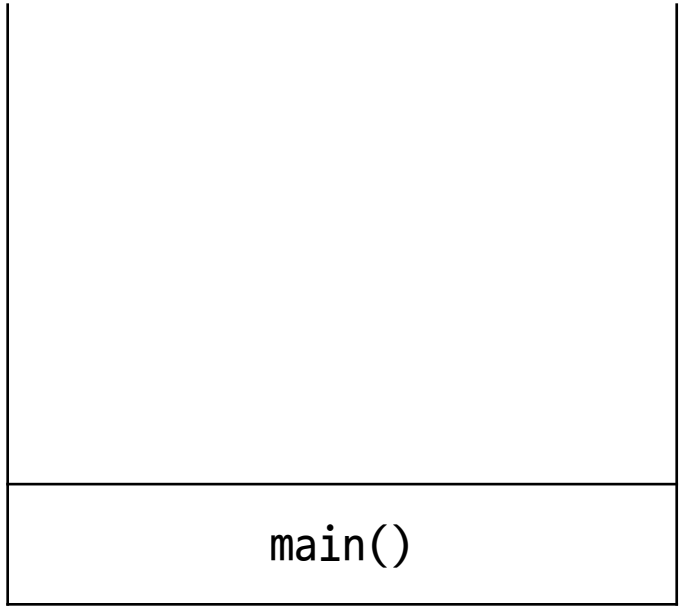
- 함수의 호출 과정

```
#include <bits/stdc++.h>

int A() { /* 구현 생략 */ }

int B() { /* 구현 생략 */ }

int main() {
    A();
    B();
}
```

함수

- 함수의 호출 과정

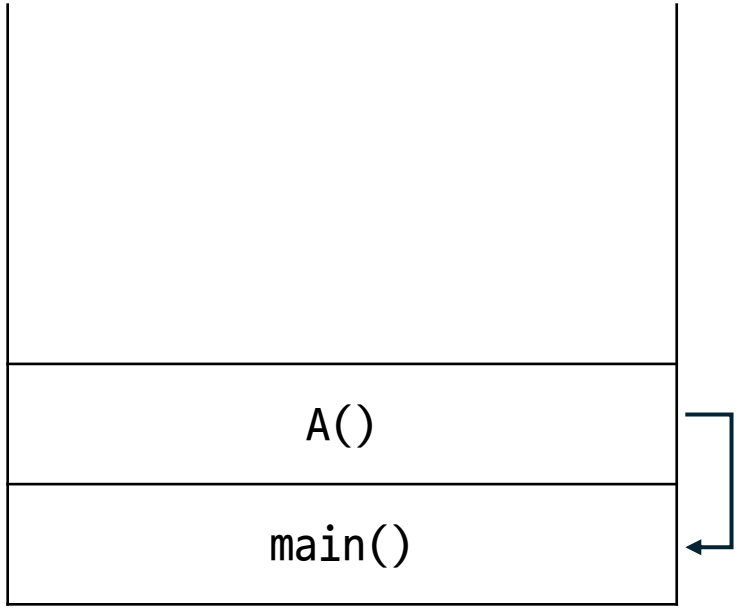
```
#include <bits/stdc++.h>

int A() { /* 구현 생략 */ }

int B() { /* 구현 생략 */ }

int main() {
    A();
    B();
}
```



함수

- 함수의 호출 과정

```
#include <bits/stdc++.h>

int A() { /* 구현 생략 */ }

int B() { /* 구현 생략 */ }

int main() {
    A();
    B();
}
```

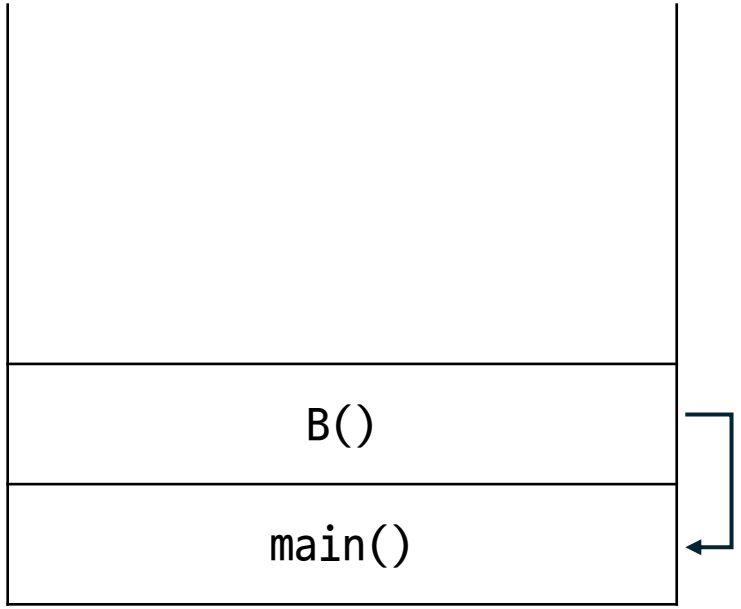


A() 종료 후 main()의 B()를 실행하는 주소 정보 저장
= A()의 return 주소에 B() 실행하는 주소가 저장됨

함수

- 함수의 호출 과정

```
#include <bits/stdc++.h>

int A() { /* 구현 생략 */ }

int B() { /* 구현 생략 */ }

int main() {
    A();
    B();
}
```

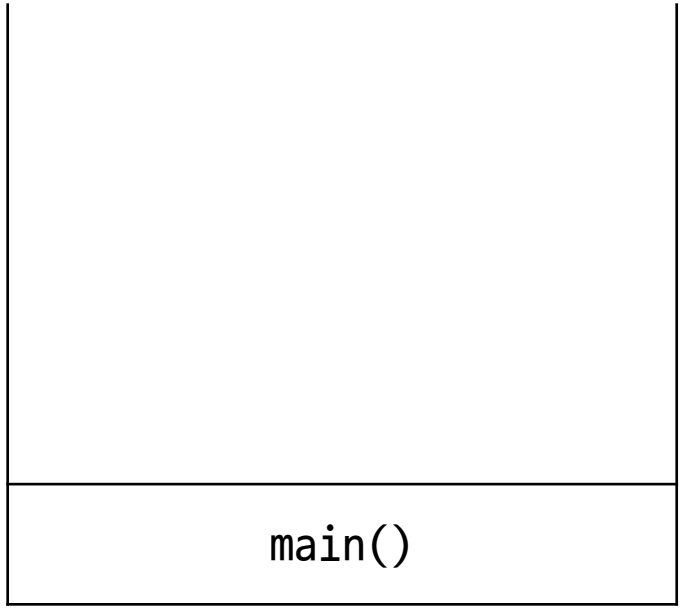


B() 종료 후 main()의 끝 주소 정보 저장
A()는 return 되었으므로 stack에서 제거

함수

- 함수의 호출 과정

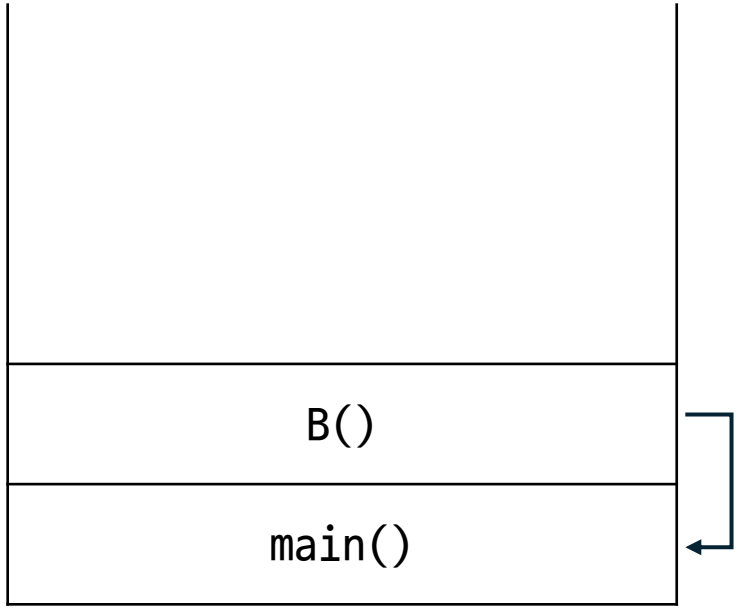
```
#include <bits/stdc++.h>

int A() { /* 구현 생략 */ }

int B() {
    A();
}

int main() {
    B();
}
```



함수

- 함수의 호출 과정

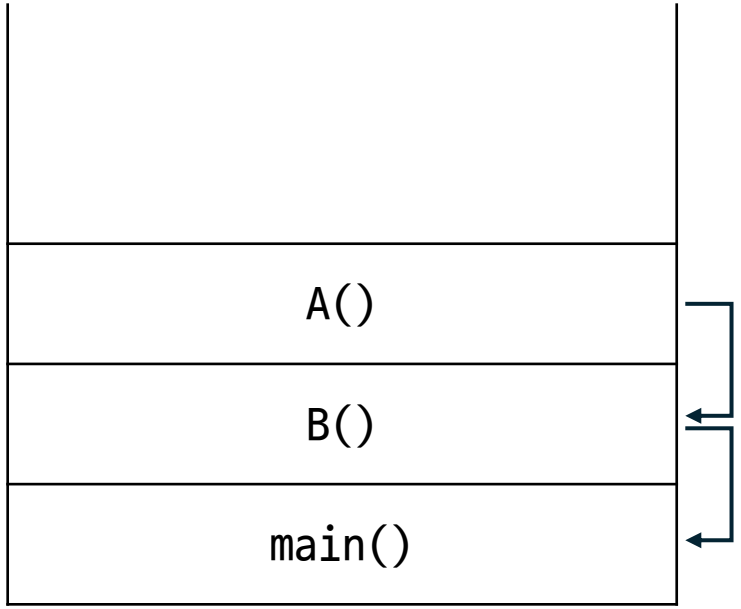
```
#include <bits/stdc++.h>

int A() { /* 구현 생략 */ }

int B() {
    A();
}

int main() {
    B();
}
```



함수

- 함수의 호출 과정

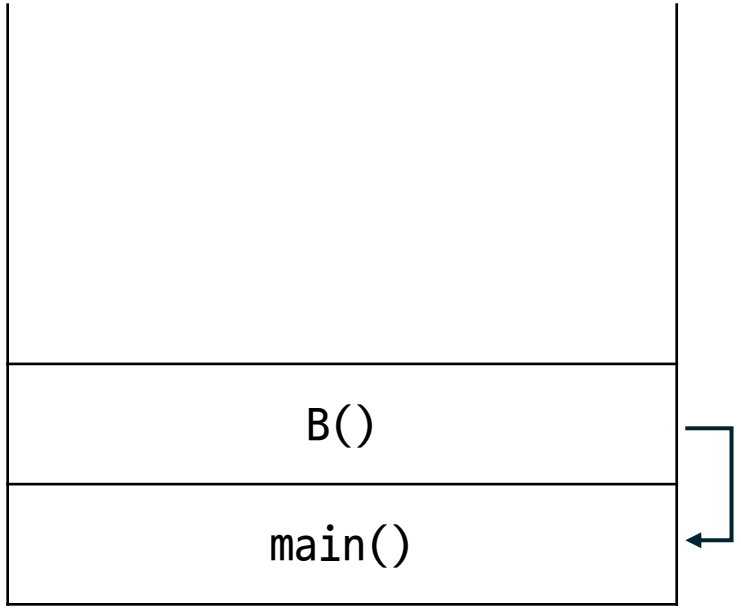
```
#include <bits/stdc++.h>

int A() { /* 구현 생략 */ }

int B() {
    A();
}

int main() {
    B();
}
```



함수

- 함수의 호출 과정

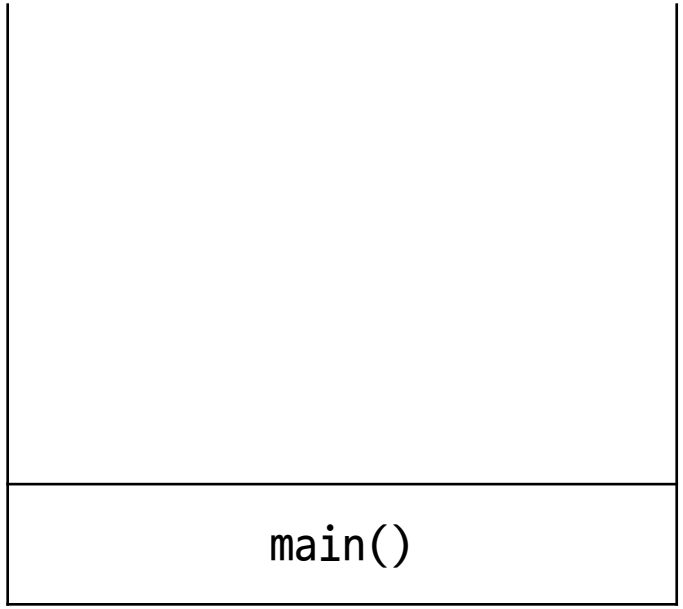
```
#include <bits/stdc++.h>

int A() { /* 구현 생략 */ }

int B() {
    A();
}

int main() {
    B();
}
```



함수

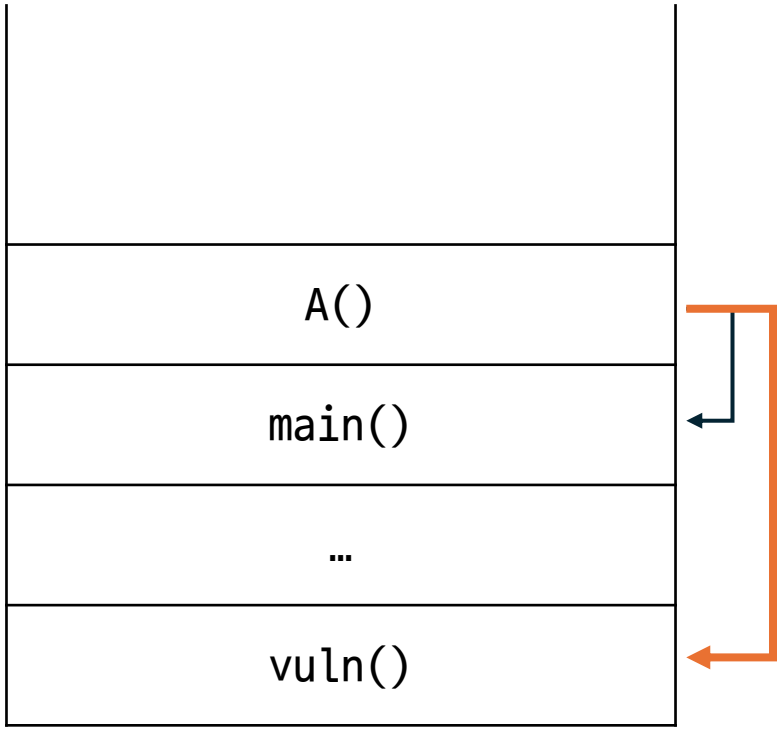
- 함수의 호출 과정

```
#include <bits/stdc++.h>

int A() { /* 구현 생략 */ }

int B() {
    A();
}

int main() {
    B();
}
```



함수

- 만약 함수의 return 주소를 덮는다면?
  - 원하는 코드를 실행할 수 있음 ➔ 해킹



함수

- 함수의 호출이 너무 많이 발생하면 컴퓨터에 할당된 stack이 초과됨
  - stack overflow

```
#include <bits/stdc++.h>

void A() {
    A();
}

int main() {
    A();
}
```

```
Process finished with exit code 139 (interrupted by signal 11:SIGSEGV)
```

함수

- 재귀함수
  - 함수 내에서 자기 자신을 호출하는 함수

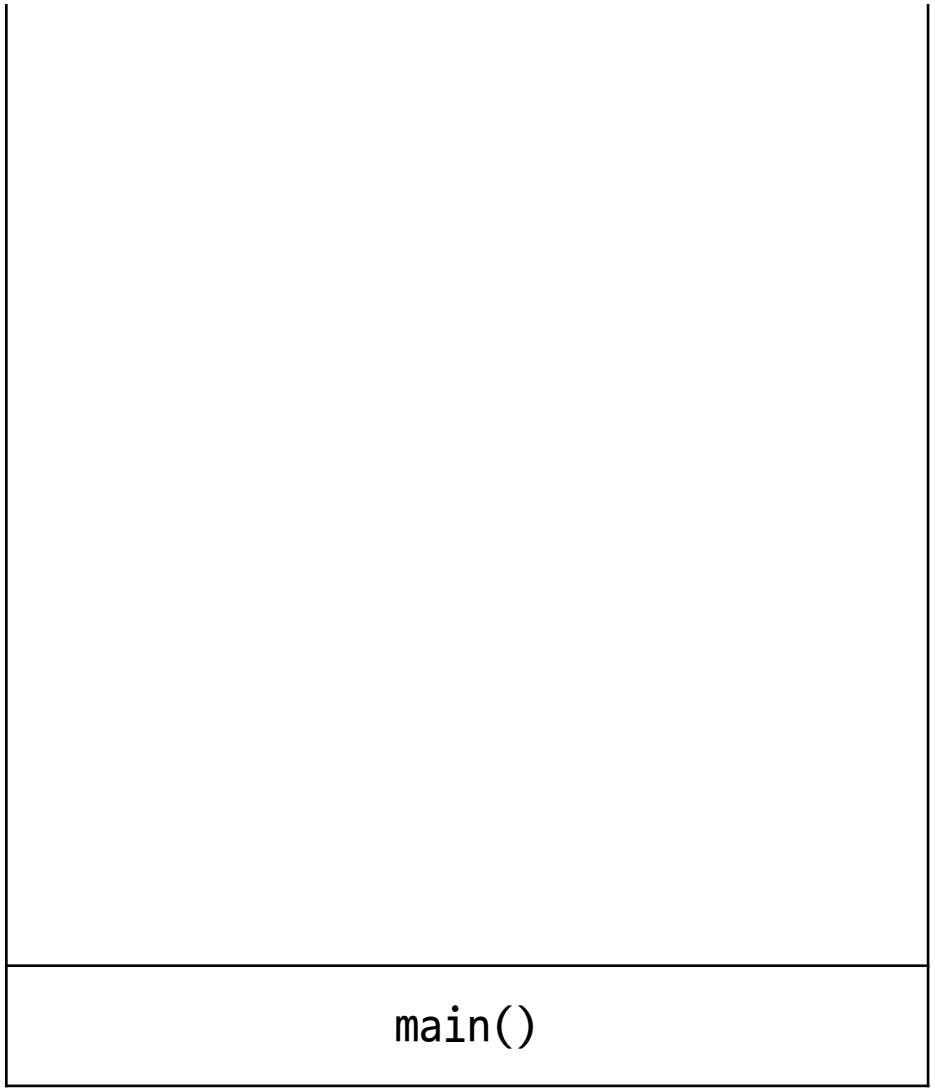
```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    printf("%d Hello World!\n", n);
    a(n-1);
}

int main() {
    a(3);
}
```

```
3 Hello World!
2 Hello World!
1 Hello World!

Process finished with exit code 0
```

함수

- 재귀함수 분석

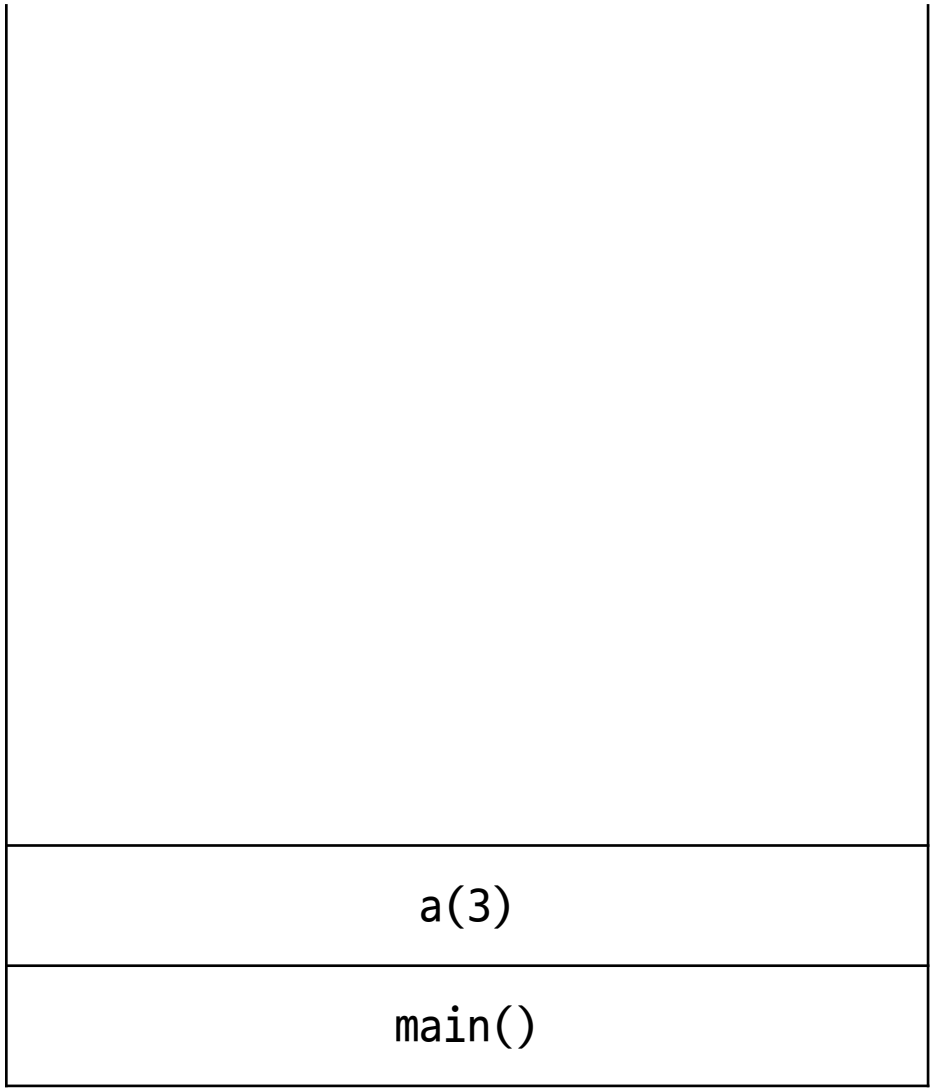
```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    printf("%d Hello World!\n", n);
    a(n-1);
}

int main() {
    a(3);
}
```



함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    printf("%d Hello World!\n", n);
    a(n-1);
}

int main() {
    a(3);
}
```



함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    printf("%d Hello World!\n", n);
    a(n-1);
}

int main() {
    a(3);
}
```

printf("%d Hello World!\n", 3)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    printf("%d Hello World!\n", n);
    a(n-1);
}

int main() {
    a(3);
}
```

a(2)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    printf("%d Hello World!\n", n);
    a(n-1);
}

int main() {
    a(3);
}
```

printf("%d Hello World!\n", 2)
a(2)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    printf("%d Hello World!\n", n);
    a(n-1);
}

int main() {
    a(3);
}
```

a(1)
a(2)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    printf("%d Hello World!\n", n);
    a(n-1);
}

int main() {
    a(3);
}
```

printf("%d Hello World!\n", 1)
a(1)
a(2)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    printf("%d Hello World!\n", n);
    a(n-1);
}

int main() {
    a(3);
}
```

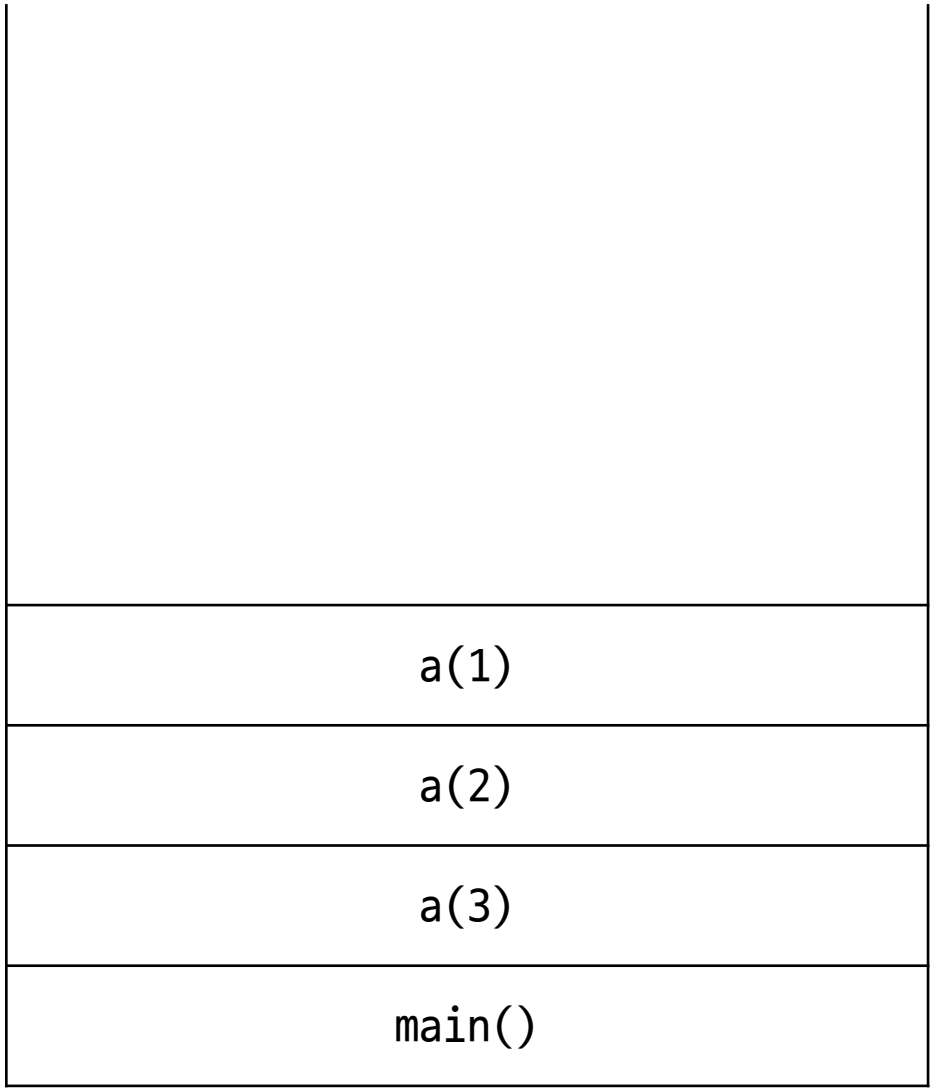
a(0)
a(1)
a(2)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    printf("%d Hello World!\n", n);
    a(n-1);
}

int main() {
    a(3);
}
```

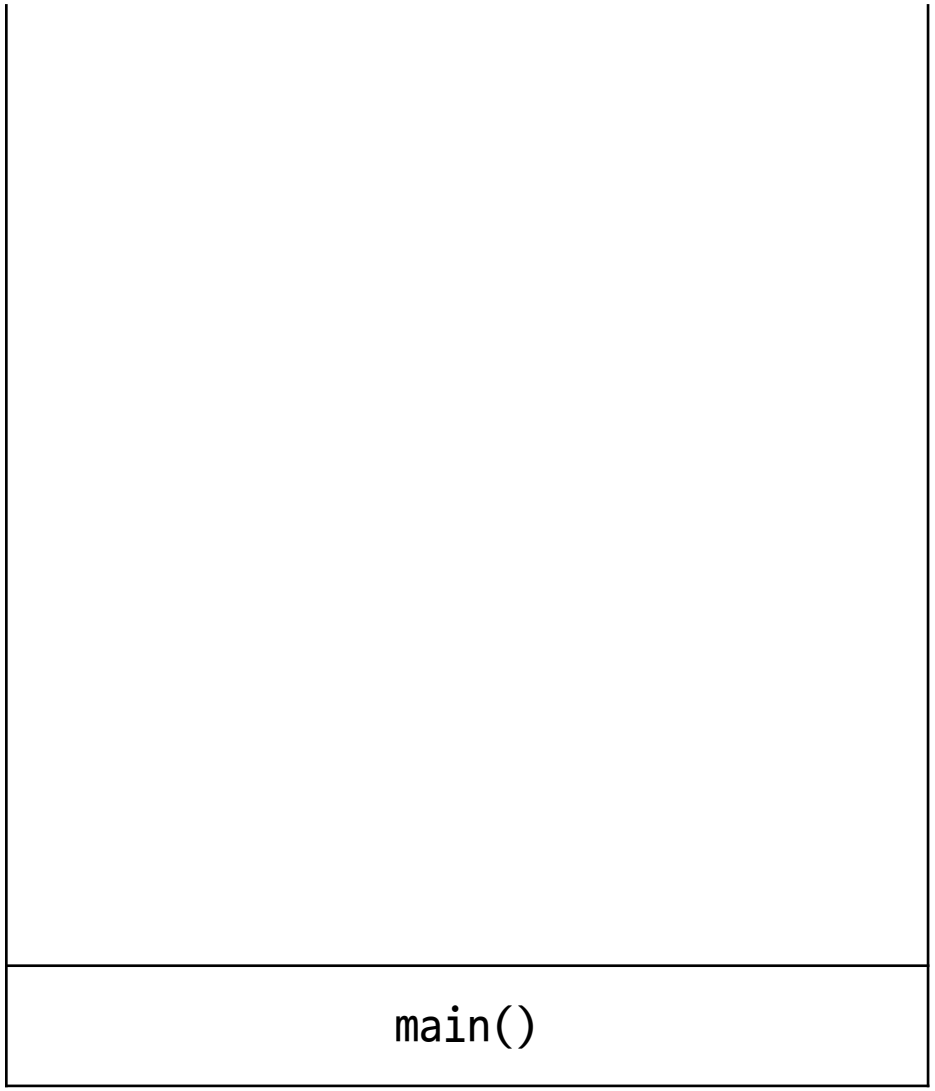
- a(1), a(2), a(3) 모두 차례대로 종료



함수

- 재귀함수 분석

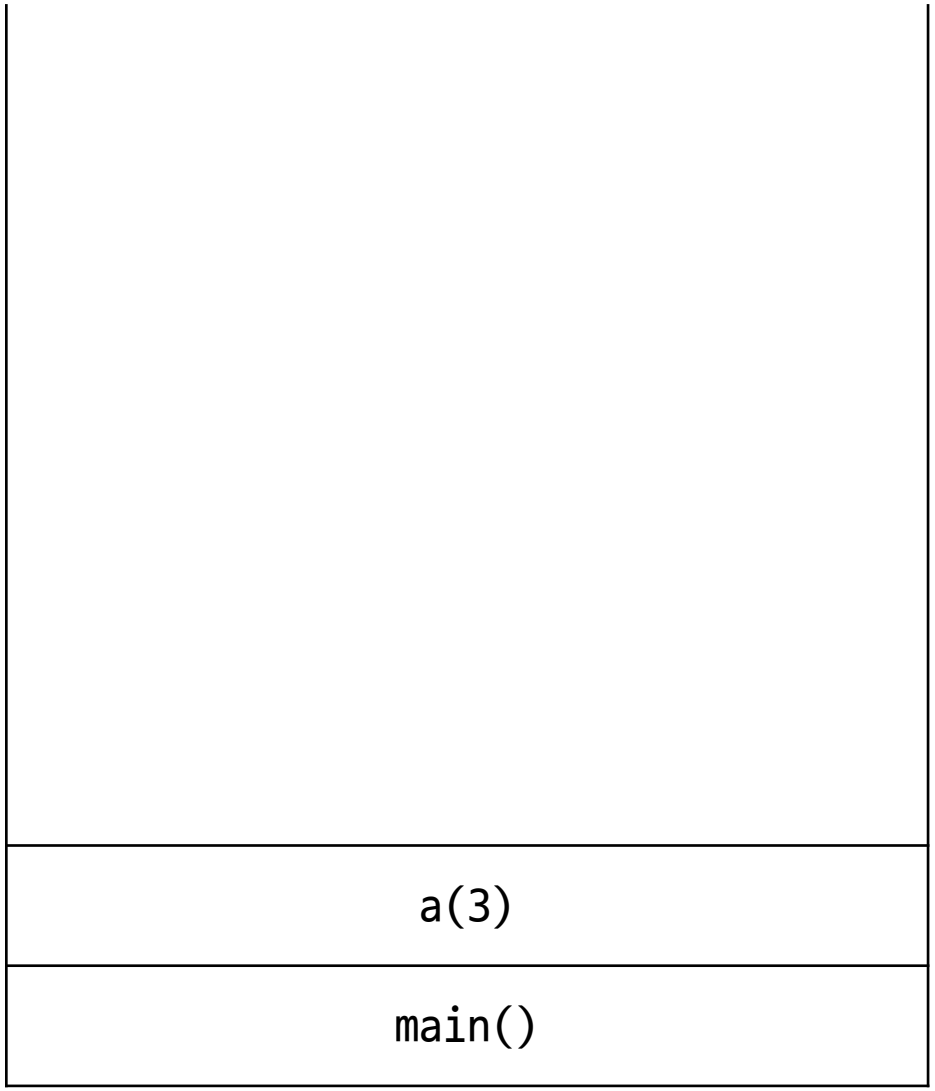
```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    a(n-1);
    printf("%d Hello World!\n", n);
}

int main() {
    a(3);
}
```



함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    a(n-1);
    printf("%d Hello World!\n", n);
}

int main() {
    a(3);
}
```



함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    a(n-1);
    printf("%d Hello World!\n", n);
}

int main() {
    a(3);
}
```

a(2)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    a(n-1);
    printf("%d Hello World!\n", n);
}

int main() {
    a(3);
}
```

a(1)
a(2)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    a(n-1);
    printf("%d Hello World!\n", n);
}

int main() {
    a(3);
}
```

a(0)
a(1)
a(2)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    a(n-1);
    printf("%d Hello World!\n", n);
}

int main() {
    a(3);
}
```

a(1)
a(2)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    a(n-1);
    printf("%d Hello World!\n", n);
}

int main() {
    a(3);
}
```

printf("%d Hello World!\n", 1)
a(1)
a(2)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    a(n-1);
    printf("%d Hello World!\n", n);
}

int main() {
    a(3);
}
```

printf("%d Hello World!\n", 1)
a(2)
a(3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    a(n-1);
    printf("%d Hello World!\n", n);
}

int main() {
    a(3);
}
```

printf("%d Hello World!\n", 2)
a(3)
main()

함수

- 재귀함수 분석

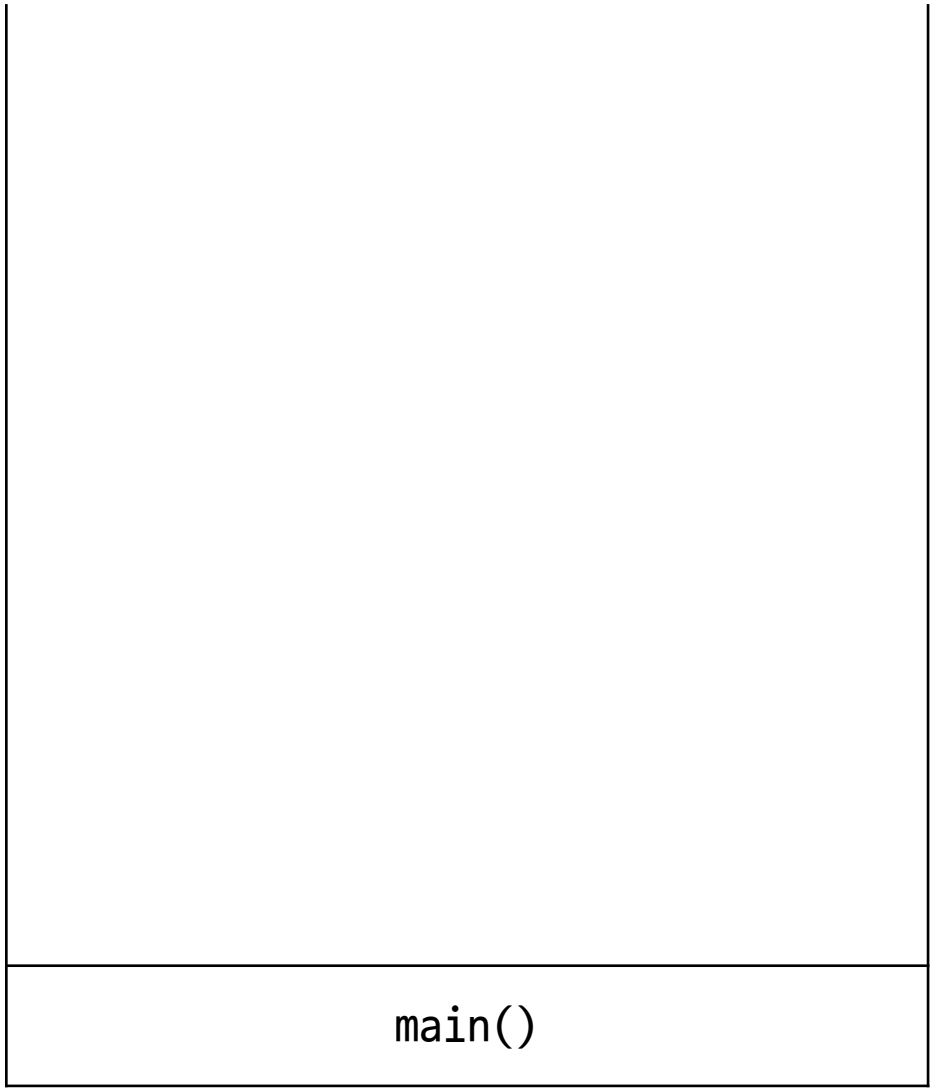
```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    a(n-1);
    printf("%d Hello World!\n", n);
}

int main() {
    a(3);
}
```

printf("%d Hello World!\n", 3)
main()

함수

- 재귀함수 분석

```
#include <bits/stdc++.h>

void a(int n) {
    if (n == 0) return;

    a(n-1);
    printf("%d Hello World!\n", n);
}

int main() {
    a(3);
}
```



함수

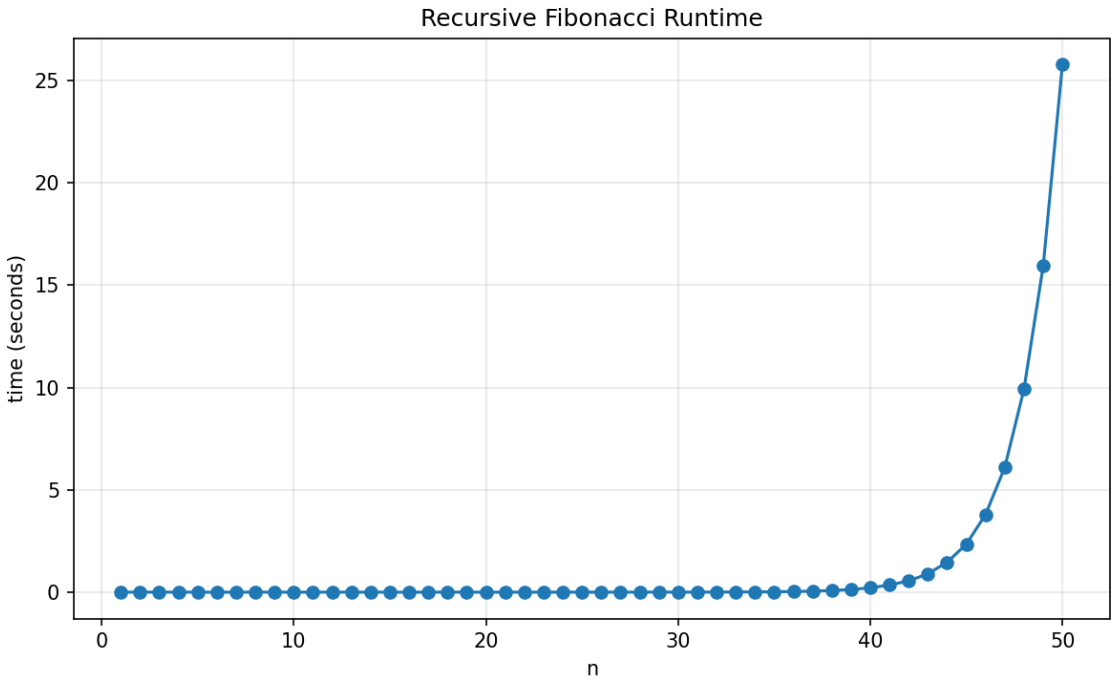
- 피보나치 재귀함수

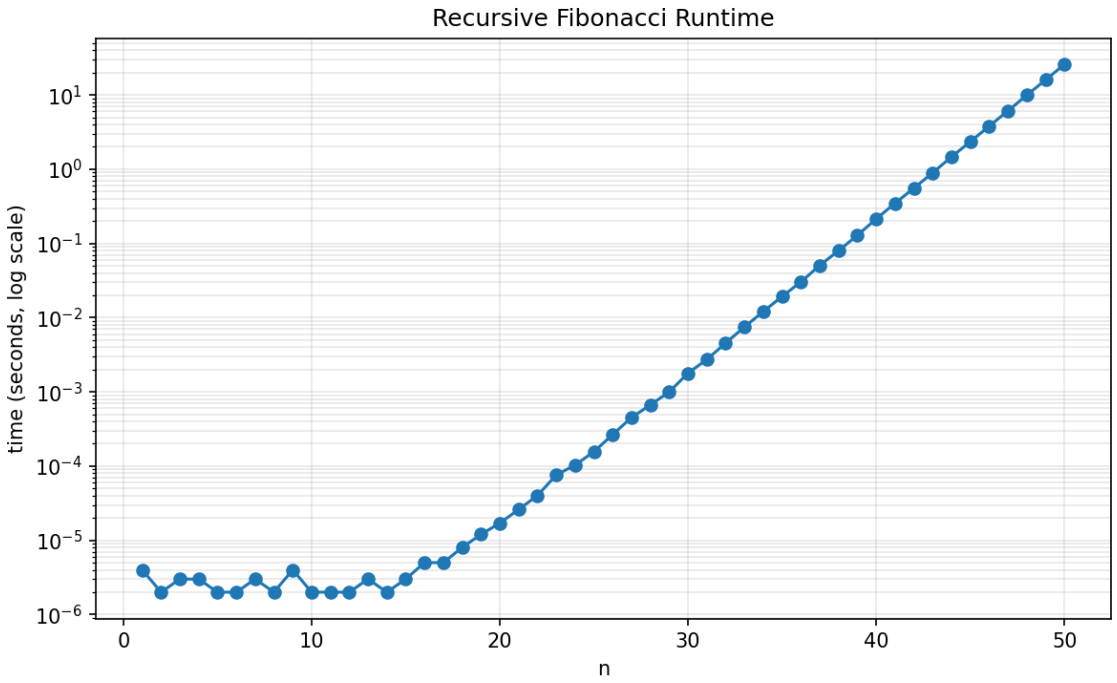
```
#include <bits/stdc++.h>

int fib(int n) {
    if (n == 1 || n == 2) {
        return 1;
    }
    return fib(n - 1) + fib(n - 2);
}

int main() {
    printf("%d\n", fib(10));
}
```

```
55
```

함수

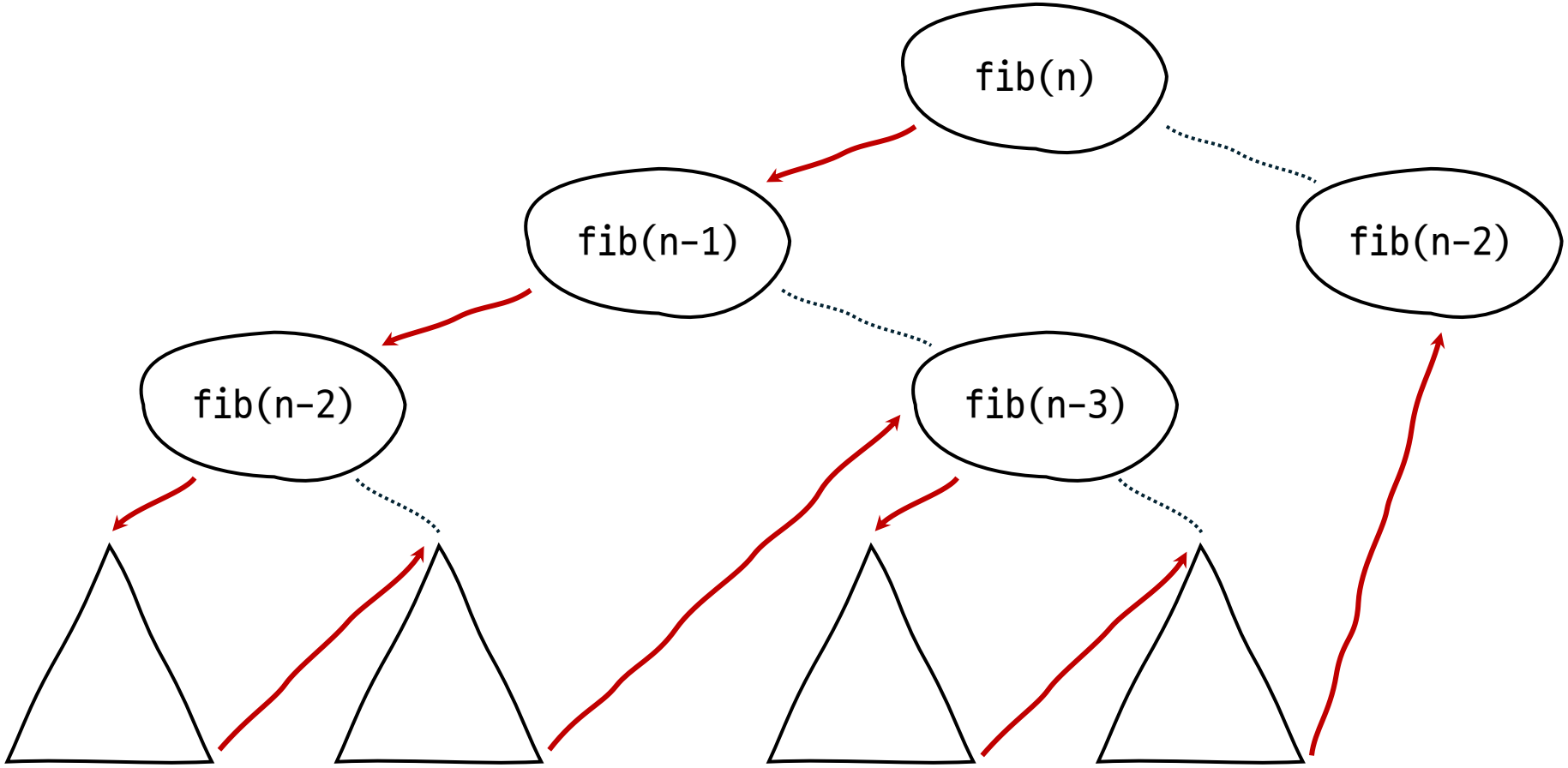
- 피보나치를 재귀적으로 돌렸을 때 걸리는 시간





함수

- n이 증가할수록 지수적으로 늘어나는 이유
  - fib(n-1) 때 구한 fib(n-2) 값을 다시 계산



함수

- 수식으로 살펴보기

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) \\
&\approx \left(\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right) \\
&\approx 1.618^n
\end{aligned}
$$

- 그러면 더 효율적으로 피보나치 수를 구하는 방법? ➔ 메모이제이션

구조체

- 여러 변수를 묶어서 저장할 경우가 필요함
  - 이름(string), 나이(int), 주소(string)

- 아래와 같이 저장할 경우 코딩이 불편해짐

```
char    name[20][20];
int     age[20];
char    address[20][100];

name[0], age[0], address[0] ➔ 0번 사람 정보
name[1], age[1], address[1] ➔ 1번 사람 정보
```

구조체

- 구조체란?
  - C언어에서 자료를 체계적으로 관리하기 위한 자료구조

- 위의 예시에서 이름, 나이, 주소를 묶어서 아래와 같은 구조체로 저장

```
struct Person {
    char name[20];
    int age;
    char address[100];
};
```

구조체

- 구조체 사용 방법

```
#include <bits/stdc++.h>

struct Person {
    char name[20];
    int age;
    char address[100];
};

int main() {
    struct Person p1, p2;

    strcpy(p1.name, "Alice");
    p1.age = 30;
    strcpy(p1.address, "123 Main St");

    p2 = {"Bob", 25, "456 Elm St"};
}
```

구조체

- 구조체 사용 방법

```
#include <bits/stdc++.h>

typedef struct Person {
    char name[20];
    int age;
    char address[100];
} p;

int main() {
    p p1, p2;

    strcpy(p1.name, "Alice");
    p1.age = 30;
    strcpy(p1.address, "123 Main St");

    p2 = {"Bob", 25, "456 Elm St"};
}
```

구조체

- 구조체 사용 방법

```
#include <bits/stdc++.h>

typedef struct {
    char name[20];
    int age;
    char address[100];
} Person;

int main() {
    Person p1, p2;

    strcpy(p1.name, "Alice");
    p1.age = 30;
    strcpy(p1.address, "123 Main St");

    p2 = {"Bob", 25, "456 Elm St"};
}
```

구조체

- 배열로도 사용 가능함

```
#include <bits/stdc++.h>

typedef struct {
    char name[20];
    int age;
    char address[100];
} Person;

int main() {
    Person p[10];

    for (int i = 0; i < 10; i++) {
        scanf("%s", p[i].name);
        scanf("%d", &p[i].age);
        scanf("%s", p[i].address);
    }
}
```

구조체의 크기

- 아래 구조체의 크기는?

```
#include <bits/stdc++.h>

typedef struct {
    char a[10];
    int b[3];
} Person;

int main() {
    printf("sizeof(Person) = %d\n", sizeof(Person));
}
```

구조체의 크기

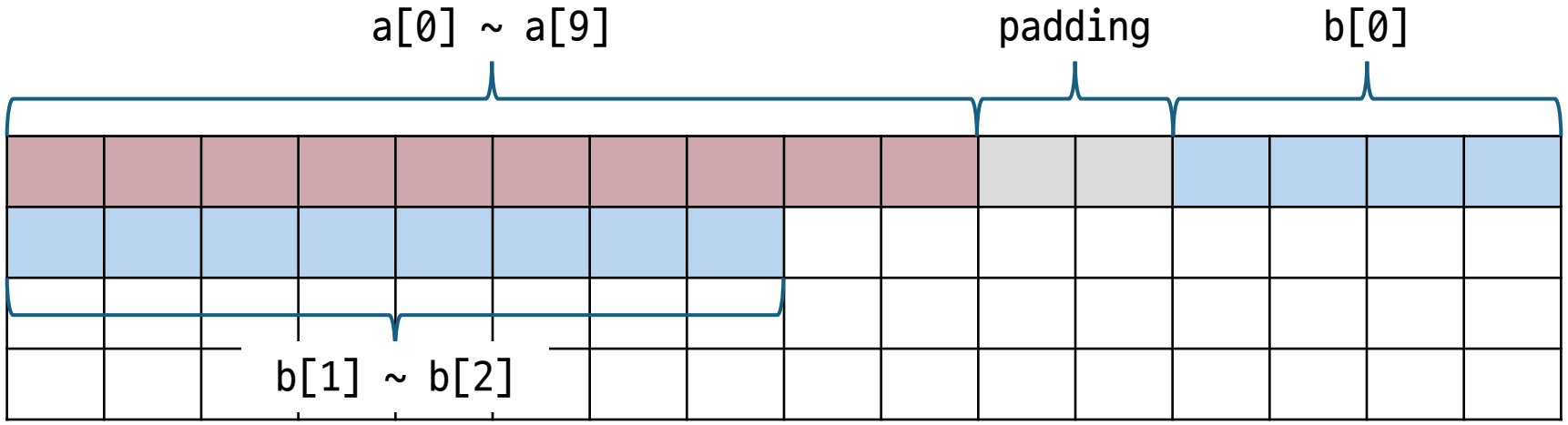
- 아래 구조체의 크기는?

```
#include <bits/stdc++.h>

typedef struct {
    char a[10];
    int b[3];
} Person;

int main() {
    printf("sizeof(Person) = %d\n", sizeof(Person));
}
```

```
sizeof(Person) = 24
```

구조체

- 구조체는 가장 큰 자료형의 배수 형태로 공간 할당

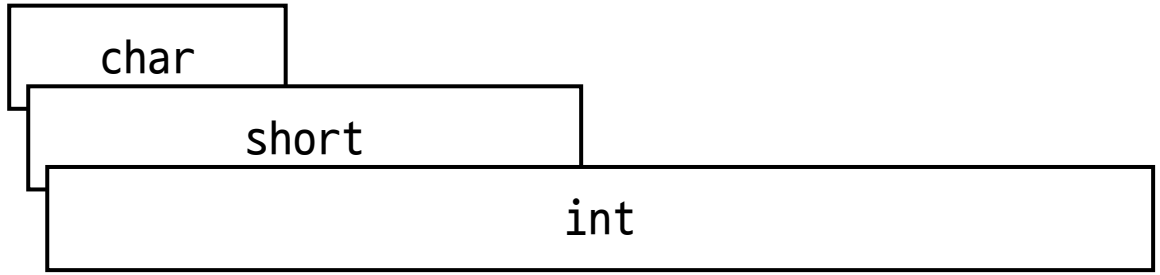


공용체

- 구조체와 비슷하지만, 모든 요소가 공간을 공유함

- 구조체 (총 8바이트)

- 공용체 (총 4바이트)



공용체

- 공용체 사용 방법

```
#include <bits/stdc++.h>

typedef union {
    char a[2];
    int b;
} Person;

int main() {
    Person p = {};
    p.a[0] = 'A';
    p.a[1] = 'B';
    printf("0x%08x\n", p.b);

    p.b = 0x41424344;
    printf("%c %c\n", p.a[0], p.a[1]);
    printf("%c %c\n", p.a[2], p.a[3]);
}
```

공용체

- 공용체 사용 방법

```
#include <bits/stdc++.h>

typedef union {
    char a[2];
    int b;
} Person;

int main() {
    Person p = {};
    p.a[0] = 'A';
    p.a[1] = 'B';
    printf("0x%08x\n", p.b);

    p.b = 0x41424344;
    printf("%c %c\n", p.a[0], p.a[1]);
    printf("%c %c\n", p.a[2], p.a[3]);
}
```

b			
a[0]	a[1]		
00	00	00	00

공용체

- 공용체 사용 방법

```
#include <bits/stdc++.h>

typedef union {
    char a[2];
    int b;
} Person;

int main() {
    Person p = {};
    p.a[0] = 'A';
    p.a[1] = 'B';
    printf("0x%08x\n", p.b);

    p.b = 0x41424344;
    printf("%c %c\n", p.a[0], p.a[1]);
    printf("%c %c\n", p.a[2], p.a[3]);
}
```

b			
a[0]	a[1]		
41('A')	42('B')	00	00

공용체

- 공용체 사용 방법

```
#include <bits/stdc++.h>

typedef union {
    char a[2];
    int b;
} Person;

int main() {
    Person p = {};
    p.a[0] = 'A';
    p.a[1] = 'B';
    printf("0x%08x\n", p.b);

    p.b = 0x41424344;
    printf("%c %c\n", p.a[0], p.a[1]);
    printf("%c %c\n", p.a[2], p.a[3]);
}
```

b			
a[0]	a[1]		
41('A')	42('B')	00	00

➔ 00004241 출력

공용체

- 공용체 사용 방법

```
#include <bits/stdc++.h>

typedef union {
    char a[2];
    int b;
} Person;

int main() {
    Person p = {};
    p.a[0] = 'A';
    p.a[1] = 'B';
    printf("0x%08x\n", p.b);

    p.b = 0x41424344;
    printf("%c %c\n", p.a[0], p.a[1]);
    printf("%c %c\n", p.a[2], p.a[3]);
}
```

b			
a[0]	a[1]		
44	43	42	41

공용체

- 공용체 사용 방법

```c
#include <bits/stdc++.h>

typedef union {
    char a[2];
    int b;
} Person;

int main() {
    Person p = {};
    p.a[0] = 'A';
    p.a[1] = 'B';
    printf("0x%08x\n", p.b);

    p.b = 0x41424344;
    printf("%c %c\n", p.a[0], p.a[1]);
    printf("%c %c\n", p.a[2], p.a[3]);
}
```

b			
a[0]	a[1]		
44	43	42	41

➔ D C 출력

공용체

- 공용체 사용 방법

```
#include <bits/stdc++.h>

typedef union {
    char a[2];
    int b;
} Person;

int main() {
    Person p = {};
    p.a[0] = 'A';
    p.a[1] = 'B';
    printf("0x%08x\n", p.b);

    p.b = 0x41424344;
    printf("%c %c\n", p.a[0], p.a[1]);
    printf("%c %c\n", p.a[2], p.a[3]);
}
```

b			
a[0]	a[1]	a[2]	a[3]
44	43	42	41

➔ B A 출력

열거형

- 정수형 상수에 이름을 붙일 때 사용

```
const int SUN = 0;
const int MON = 1;
...
const int SAT = 6;
```

```
enum Day {SUN, MON, TUE, WEN, THU, FRI, SAT};

int main() {
    Day week;
    week = MON;

    printf("%d", week); // 1
}
```

열거형

- 열거형과 정수형 차이

```
#include <bits/stdc++.h>

enum Day {SUN, MON, TUE, WEN, THU, FRI, SAT};

int main() {
    int day = MON + 1;
    printf("%d\n", day); // 2
}
```

```
#include <bits/stdc++.h>

enum Day {SUN, MON, TUE, WEN, THU, FRI, SAT};

int main() {
    Day day = MON + 1; // error, const(상수)이기 때문에 덧셈 연산 불가
    printf("%d\n", day);
}
```

열거형

- 덧셈을 하려면 아래와 같이 코딩해야 함

```
#include <bits/stdc++.h>

enum Day {SUN, MON, TUE, WEN, THU, FRI, SAT};

int main() {
    Day day = (Day) (SAT + 1);
    printf("%d\n", day); // 7
}
```

열거형

- 열거형의 초기값 지정

```
#include <bits/stdc++.h>

enum Day {SUN = 1, MON, TUE, WEN, THU, FRI, SAT};

int main() {
    Day day = TUE;
    printf("%d\n", day); // 3
}
```

```
#include <bits/stdc++.h>

enum Day {SUN, MON = 3, TUE, WEN, THU, FRI, SAT};

int main() {
    Day day1 = SUN, day2 = TUE;
    printf("%d %d\n", day1, day2); // 0, 4
}
```

열거형

- 열거형의 크기

```
#include <bits/stdc++.h>

enum Day {SUN, MON, TUE, WEN, THU, FRI, SAT};

int main() {
    printf("%d", sizeof(Day)); // 4
}
```

- 크기는 const int랑 똑같음

연습문제

- 재귀함수: 1901, 1902, 1912, 1915, 1916, 1920, 3420 (Hard)
- 구조체, 공용체, 열거형 연습문제는 없음